CS259 Mini-Project 1 — Part 1: Convolution

GPU: NVIDIA TITAN V (Volta GV100, CC 7.0)
Batch size: B = 16 (used consistently across all configurations)
Block size: 16 × 16 × 1 = 256 threads/block (K0–K4); 32 × 8 × 1 = 256 threads/block (K5)
Source files: `conv.cu` (kernels K0–K5), `conv_ref.h` (CPU reference)

Q1. Parallelization Strategy

The output tensor has four dimensions `[B][Ny][Nx][Nn]`. These are distributed across the CUDA thread hierarchy as follows for the baseline grid used by K0–K2 and K4:

Output dimension	Mapped to	Rationale
`Nn` (output channel)	`blockIdx.z`	Every thread in a block shares the same `nn`; the weight slice `weight[*][*][nn][*]` is identical for all 256 threads, enabling cooperative shared-memory load (K1, K3)
`B × Ny` (batch × row, flattened)	`blockIdx.y × TILE_Y + threadIdx.y`	Batch and spatial-y are fully independent — collapsing them into one axis maximises grid parallelism without changing the access pattern
`Nx` (spatial x)	`blockIdx.x × TILE_X + threadIdx.x`	Adjacent threads along x write to consecutive `Nn`-minor output addresses, giving coalesced global writes

Each thread's inner loop iterates over `(ky, kx, ni)` to accumulate the full dot product for its one assigned output element:

```
Grid  (K0–K2, K4): ( ceil(Nx/16),  ceil(B·Ny/16),  Nn )
                 = ( 14,           224,             64  )   — Conv1, B=16
Block (K0–K2, K4): ( 16, 16, 1 ) = 256 threads
```

K3 (`smem_wi`) — restructured grid.
The grid becomes `(ceil(Nx/16), ceil(Ny/16), B×Nn)` with `blockIdx.z = b×Nn + nn`. This guarantees every thread in a block belongs to the same batch `b`, making the 18×18 input patch contiguous in memory and loadable into shared memory without boundary discontinuities.

K5 (`warp`) — warp-level decomposition.
The flat `(ky, kx, ni)` iteration space is partitioned across 32 lanes of a warp: lane `k` handles

iterations `t = k, k+32, k+64, …`. A five-round `__shfl_down_sync` butterfly folds the 32 partial sums into lane 0. Block = `(32, 8, 1)` = 256 threads = 8 warps; each warp handles a unique `xout`.

```
Grid  (K5): ( ceil(Nx/8),  B·Ny,  Nn )
          = ( 28,          3584,   64 )   — Conv1, B=16
Block (K5): ( 32, 8, 1 ) = 256 threads
```

Limitations.

- *Grid Z saturation.* `blockIdx.z = nn` assigns one block per output channel. For Conv2 (Nn=512) this creates a 512-deep grid Z, which can leave SMs idle when the xy grid is small (only 1×14 blocks for Conv2).
- *Serial inner loop.* The `ni` loop runs serially inside each thread. For Conv2, Ni=512 means 9×512 = 4,608 FMA iterations per thread — K4 (register blocking) and K5 (warp reduction) each partially address this by restructuring how `ni` is consumed.
- *Linear scaling.* The strategy scales linearly with B, Ny, Nx, Nn. The `ni` dimension is not exposed to inter-SM parallelism, which is an opportunity for future warp-level or block-level channel reductions.

Q2. Algorithmic FLOP Count

Each output element requires $K_y \times K_x \times N_i$ fused multiply-add operations
(1 FMA = 2 FLOPs — one multiply plus one add):

$$\text{FLOPs} = 2 \times B \times N_y \times N_x \times N_n \times K_y \times K_x \times N_i$$

Conv1 (B=16, 224×224, Ni=Nn=64, Ky=Kx=3)

$$= 2 \times 16 \times 224 \times 224 \times 64 \times 3 \times 3 \times 64 = \mathbf{59.19}\ \text{GFLOPs}$$

ncu verification: `smsp__sass_thread_inst_executed_op_ffma_pred_on.sum` = 29,595,009,024
→ ×2 = **59.19 GFLOPs** ✓

Conv2 (B=16, 14×14, Ni=Nn=512, Ky=Kx=3)

$$= 2 \times 16 \times 14 \times 14 \times 512 \times 3 \times 3 \times 512 = \mathbf{14.80}\ \text{GFLOPs}$$

ncu verification: 7,398,752,256 × 2 = **14.80 GFLOPs** ✓

Q3. Execution Time and Achieved GFLOPS

Times are medians of 5 back-to-back CUDA-Event-timed runs with no profiler overhead (one warm-up run precedes the five). All kernels pass correctness verification against the OpenMP CPU reference (max absolute error < 2×10^{-9}).

Config	Kernel	Time (ms)	GFLOPS	Speedup vs naive
Conv1	K0 naive	686.4	86.2	1.00×
224×224	K1 smem_w	665.7	88.9	1.03×
Ni=Nn=64	K2 unroll	1128.6	52.4	0.61× ↓
	K3 smem_wi	**57.0**	**1037.6**	**12.0× ↑**
	K4 regblock	397.2	149.0	1.73×
	K5 warp	171.0	346.1	4.01×
Conv2	K0 naive	84.2	175.7	1.00×
14×14	K1 smem_w	99.2	149.2	0.85× ↓
Ni=Nn=512	K2 unroll	86.9	170.3	0.97×
	K3 smem_wi	**12.5**	**1181.5**	**6.73× ↑**
	K4 regblock	23.4	631.5	3.59×
	K5 warp	39.3	376.8	2.15×

TITAN V peak FP32: 14,900 GFLOPS.
Best achieved: Conv2 K3 smem_wi at 1181.5 GFLOPS = **7.9% of peak**.
Conv1 K3 smem_wi at 1037.6 GFLOPS = **7.0% of peak**.
Both naive kernels are below 1.2% of peak, confirming severe memory bottlenecks.

Q4. Roofline Analysis

Hardware — NVIDIA TITAN V

Parameter	Value
Peak FP32 FLOPS	14,900 GFLOPS (14.9 TFLOPS)
Peak HBM2 bandwidth	652.8 GB/s
Ridge point	**22.82 FLOPs/byte**
L2 cache	4.5 MB
Shared memory / SM	96 KB
CUDA cores	5120
SM count	80

> **Unit note.** ncu reports `dram__bytes_read.sum` and `dram__bytes_write.sum` in `Gbyte` = 10^9 bytes (SI, not 2^{30}). Cross-check: Conv1 naive reads 420.65 × 10^9 bytes in 686 ms → 613 GB/s = 93.9% × 652.8 GB/s, matching ncu's `gpu__dram_throughput = 94.0%` to within 0.1%. ✓

Theoretical Arithmetic Intensity

The theoretical AI assumes every tensor byte is read or written exactly once (no re-fetching):

$$\text{AI}_{\text{theory}} = \frac{\text{algorithmic FLOPs}}{\text{bytes(synapse)} + \text{bytes(neuron\_i)} + \text{bytes(neuron\_n)}}$$

Conv1 — minimum memory footprint:

Tensor	Dimensions	Size
synapse	[3][3][64][64]	147,456 B (0.14 MB)
neuron_i	[16×226][226][64]	208,949,248 B (199.3 MB)
neuron_n	[16×224][224][64]	205,520,896 B (196.0 MB)
Total minimum		**414,617,600 B (395.5 MB)**

$$\text{AI}_{\text{theory,Conv1}} = \frac{59.19 \times 10^9}{414.6 \times 10^6} = \mathbf{142.7}\ \text{FLOPs/byte}$$

Conv2 — minimum memory footprint:

Tensor	Dimensions	Size
synapse	[3][3][512][512]	9,437,184 B (9.0 MB)
neuron_i	[16×16][16][512]	8,388,608 B (8.0 MB)
neuron_n	[16×14][14][512]	6,422,528 B (6.1 MB)
Total minimum		**24,248,320 B (23.1 MB)**

$$\text{AI}_{\text{theory,Conv2}} = \frac{14.80 \times 10^9}{24.25 \times 10^6} = \mathbf{610.2}\ \text{FLOPs/byte}$$

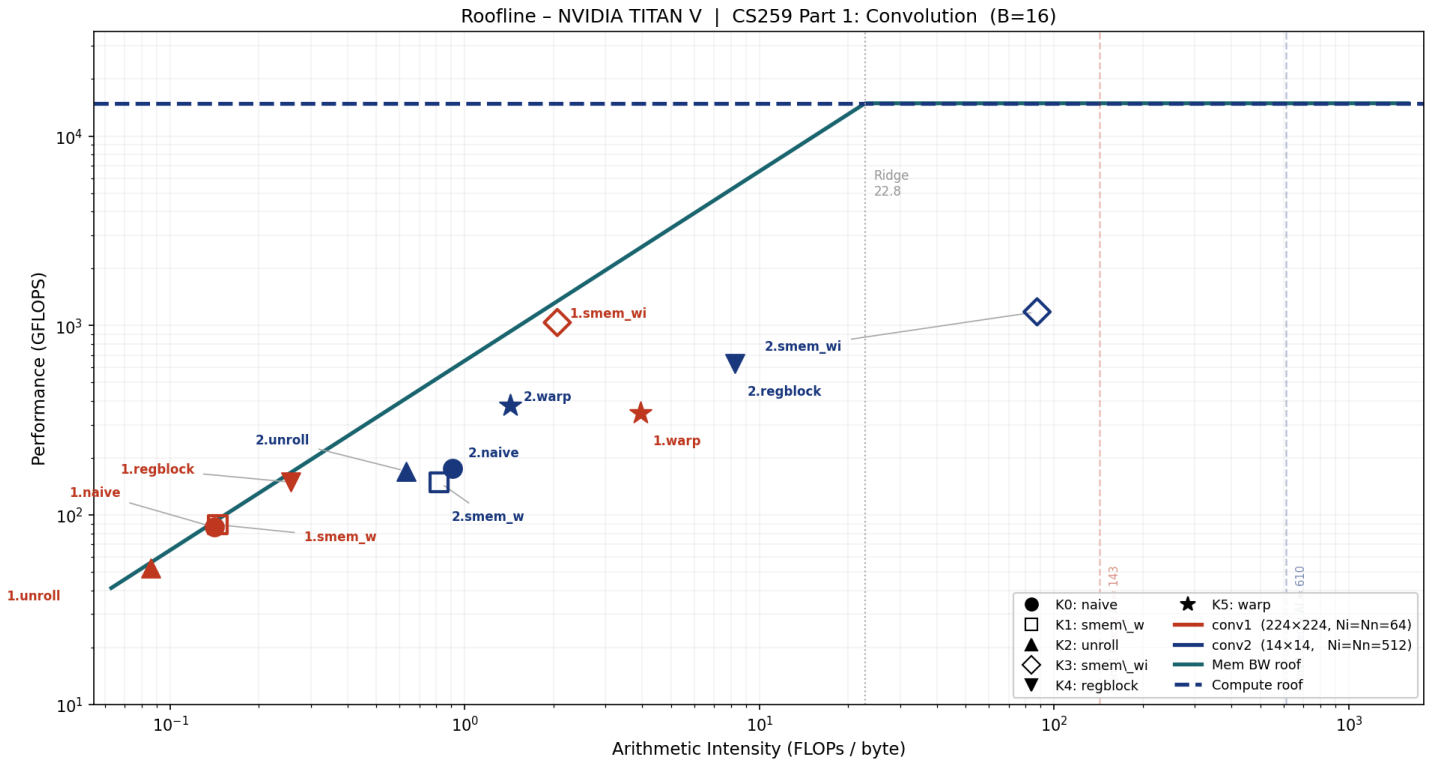
Both values far exceed the ridge point (22.82), so **both configurations are theoretically compute-bound** if every DRAM byte is maximally reused.

Measured Arithmetic Intensity

$$\text{AI}_{\text{measured}} = \frac{\texttt{smsp\_\_sass\_thread\_inst\_executed\_op\_ffma\_pred\_on.sum} \times 2}{\texttt{dram\_\_bytes\_read.sum} + \texttt{dram\_\_bytes\_write.sum}}$$

Config	Kernel	DRAM Read (GB)	DRAM Write (GB)	Total DRAM (GB)	Actual AI	DRAM%	SM%	Occ%	Bound
Conv1	K0 naive	419.00	1.65	420.65	0.141	94.0	2.82	73.2	**Memory**
	K1 smem_w	406.82	1.65	408.47	0.145	94.2	2.91	73.3	**Memory**
	K2 unroll	689.25	1.65	690.90	0.086	93.6	1.70	97.4	**Memory**
	K3 smem_wi	27.10	1.64	28.74	2.059	77.2	44.1	37.3	**Memory**
	K4 regblock	229.89	0.41	230.30	0.257	89.4	3.05	49.4	**Memory**
	K5 warp	13.39	1.65	15.04	3.936	13.5	73.4	70.9	**Memory**
Conv2	K0 naive	16.26	0.045	16.31	0.908	29.4	6.47	73.8	**Memory**
	K1 smem_w	18.06	0.047	18.11	0.817	27.7	5.52	61.5	**Memory**
	K2 unroll	23.32	0.046	23.37	0.633	40.1	6.14	97.6	**Memory**
	K3 smem_wi	0.162	0.008	0.169	**87.49**	2.07	60.0	37.4	**Compute**
	K4 regblock	1.780	0.012	1.792	8.258	11.2	14.5	48.0	**Memory**
	K5 warp	10.30	0.053	10.35	1.429	40.3	70.9	59.9	**Memory**

Roofline Chart



Filled markers = K0/K2/K4/K5 (naive/unroll/regblock/warp); hollow markers = K1/K3 (smem_w/smem_wi). Red = Conv1; blue = Conv2. Dashed vertical lines = theoretical minimum AI. Dotted vertical = ridge point (22.82).

Gap Between Theoretical and Measured AI

Config	Theory AI	Best measured AI	DRAM excess factor
Conv1 naive	142.7	0.141	**×1012**
Conv1 K3 smem_wi	142.7	2.059	×69
Conv2 naive	610.2	0.908	**×672**
Conv2 K3 smem_wi	610.2	**87.49**	×7

Conv1 analysis. The 144 KB weight tensor fits within TITAN V's 4.5 MB L2 cache, but 51.4 M output elements each independently re-fetch 576 weight values. Concurrent warp pressure evicts weight cache lines before they can be reused, causing most reads to reach DRAM. The 199 MB padded input is too large to cache at all. Even the best Conv1 kernel (K3 smem_wi) brings actual AI only to 2.06 — still well below the ridge point — because the data movement is now dominated by L1/L2 rather than DRAM, but the kernel is not yet compute-bound.

Conv2 analysis. The padded input (8 MB) and output (6 MB) both fit in L2, and `l1tex__t_bytes_pipe_lsu_mem_global_op_ld = 245 GB` vs `dram__bytes_read = 16.3 GB` confirms ~94% of L1 misses are served by L2 for the naive kernel. The 9.4 MB weight is larger than L2, so repeated weight fetches partially escape to DRAM. K3 smem_wi tiles the weight into 8-channel shared-memory chunks, collapsing DRAM traffic to 162 MB and

raising actual AI to **87.49 FLOPs/byte** — the **only kernel in this study to exceed the ridge point (22.82)** and enter the compute-bound regime. SM throughput reaches 60%, up from 6.5% for naive.

Q5. Optimisations

Six kernels were implemented, profiled, and compared against the naive baseline. Each targets a different source of inefficiency identified from the Roofline analysis.

K1 — Shared-Memory Weight Tile (`smem_w`)

Idea. Every thread in a block shares `blockIdx.z = nn`, so all 256 threads need the identical weight slice `weight[*][*][nn][*]`. Load this slice cooperatively into shared memory once per block; subsequent accesses hit SRAM (~20 ns) rather than DRAM (~600 ns).

Shared memory per block = Ky × Kx × Ni × 4 bytes:

- Conv1 (Ni=64): **2.25 KB** — thread count limits the SM to 8 blocks/SM regardless; no occupancy penalty.
- Conv2 (Ni=512): **18.0 KB** — smem-limited: 96 KB ÷ 18 KB = 5.3 → **5 blocks/SM < 8** → occupancy drops from 73.8% to 61.5%.

Conv1 results:

Metric	naive	smem_w	Δ
Time (ms)	686.4	665.7	–3.1%
GFLOPS	86.2	88.9	+3.2%
DRAM total (GB)	420.65	408.47	–2.9%
Actual AI	0.141	0.145	+2.8%
DRAM throughput	94.0%	94.2%	≈ same
Occupancy	73.2%	73.3%	≈ same

The improvement is marginal. Weight DRAM traffic drops by 12 GB, but the kernel is dominated by the 199 MB padded input which is too large for L2 and is unaffected by smem_w. DRAM throughput stays at ~94%, confirming that weight was not the primary bottleneck. The 2.25 KB smem tile for Conv1 is small enough that the weight was already partially served by L2 in the naive kernel.

Conv2 results:

Metric	naive	smem_w	Δ
Time (ms)	84.2	99.2	**+17.8%** ↑ slower
GFLOPS	175.7	149.2	–15.1%
DRAM total (GB)	16.31	18.11	+11.0%
Actual AI	0.908	0.817	–10.0%
L2 miss traffic (GB)	37.44	87.35	**+133%**
Occupancy	73.8%	61.5%	–16.7%

smem_w is **counterproductive** for Conv2. The 18 KB tile reduces concurrent blocks/SM from 8 to 5, cutting occupancy by 16.7%. Fewer resident warps means fewer in-flight memory requests, degrading L2 cache-line utilisation and causing 133% more L2 misses to reach DRAM — DRAM traffic increases rather than decreasing. The smem tile for Conv2 is too large to fit within the shared-memory budget without sacrificing occupancy.

K2 — Loop Unrolling (`unroll`)

Idea. `KY = KX = 3` are compile-time constants. Applying `#pragma unroll` fully unrolls those two loops into 9 independent FMA chains, eliminating loop-control instructions (compare, increment, branch) and allowing the compiler to expose instruction-level parallelism across all 9 filter positions simultaneously. The `ni` loop is runtime-dynamic and is not unrolled.

Conv1 results:

Metric	naive	unroll	Δ
Time (ms)	686.4	1128.6	**+64.4%** ↑ slower
GFLOPS	86.2	52.4	–39.2%
DRAM total (GB)	420.65	690.90	**+64.2%**
L2 misses (GB)	246.5	384.3	**+55.9%**
Occupancy	73.2%	**97.4%**	+33%
SM throughput	2.82%	1.70%	–40%

Severe regression despite higher occupancy. Unrolling generates 9 independent load address streams — one per (ky, kx) — that the scheduler issues simultaneously. These 9 streams target different cache sets in L2, causing cache-line thrashing: L2 misses grow by +138 GB and DRAM traffic by +270 GB. The SM is busy issuing loads (occupancy 97%) but stalls waiting for them to return (SM throughput 1.70%). Occupancy rises because unrolling

removes loop counter registers, reducing per-thread register pressure and allowing more warps per SM — but those warps are all stalled on the same L2 bottleneck.

Conv2 results:

Metric	naive	unroll	Δ
Time (ms)	84.2	86.9	+3.2%
GFLOPS	175.7	170.3	–3.1%
DRAM total (GB)	16.31	23.37	+43.3%
Occupancy	73.8%	97.6%	+32%

Near-neutral for Conv2. DRAM traffic still increases (+43%) but Conv2's smaller spatial extent limits the absolute penalty. The same occupancy paradox appears (97.6%), confirming the mechanism is identical.

Conclusion. Loop unrolling is harmful for memory-bound kernels: it amplifies the number of concurrent load streams, overwhelming L2. It would be beneficial only when the kernel is compute-bound or when the unrolled iterations access the same cache lines.

K3 — Shared-Memory Weight + Input Tile (`smem_wi`)

Idea. Extend K1 by also tiling the input into shared memory. Processing `ni` in chunks of `SMWI_TILE_NI = 8` channels at a time, each iteration:

1. Cooperatively loads `weight[ky][kx][nn][ni_chunk]` → `smem_w` (288 bytes)
2. Cooperatively loads the 18×18 input patch `input[b][yout_base±1][xout_base±1][ni_chunk]` → `smem_i` (10,368 bytes)
3. Computes the 16×16 output tile entirely from shared memory.

The 18×18 patch covers the full neighbourhood needed by any of the 16×16 output threads using a 3×3 filter. Each input byte is fetched from DRAM once and reused 256 times within the block. Total smem = (9×8 + 18×18×8) × 4 = **10,656 bytes (10.4 KB)** per block.
At 96 KB per SM: 96/10.4 ≈ 9 blocks — but the thread-count limit (2048/256 = 8) binds first, so **occupancy is unchanged from the smem-budget perspective**.

The grid is restructured to `blockIdx.z = b × Nn + nn` so all threads in a block share the same batch index, keeping the input patch contiguous in memory.

Conv1 results:

Metric	naive	smem_wi	Δ
Time (ms)	686.4	57.0	**–91.7%**
GFLOPS	86.2	1037.6	**+12.0×**

Metric	naive	smem_wi	Δ
DRAM total (GB)	420.65	28.74	**–93.2%**
Actual AI	0.141	2.059	**+14.6×**
DRAM throughput	94.0%	77.2%	–17 pp
SM throughput	2.82%	44.1%	**+15.6×**
L1 requests (GB)	969.8	16.6	**–98.3%**
Occupancy	73.2%	37.3%	–50%

The 12× speedup comes from a 93% reduction in DRAM traffic. Each 18×18×8 input patch (41 KB) is loaded once from DRAM and shared across all 256 threads and all 9 filter positions, achieving 16×16 output reuse per input load. SM throughput jumps from 2.82% to 44.1%. Occupancy falls to 37% — the smem allocation is not the cause (10.4 KB leaves headroom), but the restructured grid (grid.z = B×Nn = 1024) creates fewer blocks per SM on average. Nevertheless, the reduced memory-stall time dominates.

Conv2 results:

Metric	naive	smem_wi	Δ
Time (ms)	84.2	12.5	**–85.1%**
GFLOPS	175.7	1181.5	**+6.73×**
DRAM total (GB)	16.31	0.169	**–98.9%**
Actual AI	0.908	**87.49**	**+96×**
DRAM throughput	29.4%	2.07%	–93 pp
SM throughput	6.47%	60.0%	**+9.3×**

Conv2 smem_wi is the **only kernel in this study to cross the ridge point (22.82)**, achieving an actual AI of 87.49 FLOPs/byte. DRAM traffic collapses to 162 MB: the 9.4 MB weight is loaded in 8-channel tiles that are fully reused across all 14×14 = 196 spatial positions, and the 8 MB input (which fits in L2) contributes almost nothing to DRAM reads. SM throughput reaches 60%, confirming a genuine transition from memory-bound to compute-bound operation.

K4 — Register Blocking (`regblock`)

Idea. Each thread computes `REG_N = 4` consecutive output channels (`nn_base` to `nn_base+3`) simultaneously. The input value `inp[b][y+ky][x+kx][ni]` is read once from memory into a register (`in_val`), then multiplied against 4 different weight values. This amortises the input memory traffic by 4× in principle.

Grid Z = ⌈Nn/4⌉ instead of Nn; everything else in the grid is unchanged.

Conv1 results:

Metric	naive	regblock	Δ
Time (ms)	686.4	397.2	**–42.1%**
GFLOPS	86.2	149.0	**+1.73×**
DRAM total (GB)	420.65	230.30	**–45.2%**
Actual AI	0.141	0.257	+82%
Occupancy	73.2%	49.4%	–33%

Input DRAM reads drop from ~420 GB to ~230 GB. The savings fall short of the theoretical 4× because each thread must also read 4× as many weight values (one per nn), partially offsetting the gain. Occupancy drops from 73% to 49% because 4 accumulator registers per thread increase register pressure. The 1.73× speedup confirms that input reuse is more valuable than the occupancy cost in this regime.

Conv2 results:

Metric	naive	regblock	Δ
Time (ms)	84.2	23.4	**–72.2%**
GFLOPS	175.7	631.5	**+3.59×**
DRAM total (GB)	16.31	1.792	**–89.0%**
Actual AI	0.908	8.258	**+9.1×**

Conv2 benefits more (3.6× vs 1.7×) because the 4 consecutive `nn` values access weight data that is already resident in L2 from the previous iteration, giving near-free weight reuse on top of the 4× input reuse. Actual AI rises from 0.91 to 8.26 — approaching but still below the ridge point.

K5 — Warp-Level ni Reduction (`warp`)

Idea. An entire 32-lane warp collaborates on one output element. The flat `(ky, kx, ni)` space (Conv1: 9×64 = 576 iterations; Conv2: 9×512 = 4,608) is striped across lanes: lane `k` handles `t = k, k+32, k+64, …`. After accumulation, five rounds of `__shfl_down_sync` fold the 32 partial sums into lane 0, which writes the output — with no shared memory involved.

Memory coalescing improvement. In the naive kernel, adjacent threads differ in `xout`, so they access `inp[...][xout][ni]` with a stride of `Ni × 4 = 256 bytes` between neighbours — 64 cache lines for 32 threads. In K5, adjacent lanes differ only in `ni`:

`inp[...][same_xout][ni]` advances by 4 bytes between lanes. The entire warp's 32 accesses fit in one 128-byte cache line.

Conv1 results:

Metric	naive	warp	Δ
Time (ms)	686.4	171.0	**–75.1%**
GFLOPS	86.2	346.1	**+4.01×**
DRAM total (GB)	420.65	15.04	**–96.4%**
Actual AI	0.141	3.936	**+27.9×**
DRAM throughput	94.0%	13.5%	–86 pp
SM throughput	2.82%	73.4%	**+26×**
L1 requests (GB)	969.8	236.8	–75.6%

DRAM drops from 420 GB to 15 GB — a 28× reduction driven entirely by coalescing: the naive kernel wasted 63 of every 64 cache-line bytes fetched for input; the warp kernel uses all 32 floats in each line. SM throughput jumps to 73.4%, the second-highest of all kernels.

Conv2 results:

Metric	naive	warp	Δ
Time (ms)	84.2	39.3	**–53.3%**
GFLOPS	175.7	376.8	**+2.15×**
DRAM total (GB)	16.31	10.35	–36.5%
Actual AI	0.908	1.429	+57%
SM throughput	6.47%	70.9%	**+10.9×**

The DRAM reduction is more modest (37%) because Conv2's input was already partially cached in L2 for the naive kernel. SM throughput still rises to 71%, confirming genuine compute activity. The 2.15× speedup is smaller than Conv1's 4× because the coalescing benefit is smaller when cache pressure is lower.

Summary

Kernel	Technique	Conv1 vs naive	Conv2 vs naive	Key tradeoff
K1 smem_w	Shared weight tile	+3%	**–15%**	2.25 KB smem: no penalty for Conv1; 18 KB smem cuts Conv2 occupancy

Kernel	Technique	Conv1 vs naive	Conv2 vs naive	Key tradeoff
				16%
K2 unroll	`#pragma unroll` (ky/kx)	**–39%**	–3%	9 simultaneous load streams thrash L2; DRAM +64% for Conv1
K3 smem_wi	Shared weight + input tile	**+12.0×**	**+6.73×**	Best overall; 10.4 KB smem; no occupancy penalty; DRAM –93%/ –99%
K4 regblock	REG_N=4 output channels/thread	+1.73×	+3.59×	Input reads ÷4; register pressure –33% occupancy
K5 warp	Warp ni-reduction + coalescing	+4.01×	+2.15×	Stride-1 ni access; DRAM –96% Conv1; no smem; shfl reduction overhead

Most impactful. K3 smem_wi — 12× (Conv1) and 6.7× (Conv2). Tiling both weight and input into shared memory simultaneously eliminates the two dominant DRAM bottlenecks. Conv2 smem_wi is the only kernel to enter the compute-bound regime (actual AI 87.49 > ridge point 22.82).

Counterproductive optimisations. K2 (unroll) regresses Conv1 by 39% due to L2 cache thrashing from 9 simultaneous load streams. K1 (smem_w) regresses Conv2 by 15% because the 18 KB tile reduces SM occupancy and paradoxically increases DRAM traffic.

Insight. Conv1 and Conv2 have opposite characteristics: Conv1 has a large spatial footprint (224×224) but a small weight (144 KB), making spatial input reuse the key lever. Conv2 has a tiny spatial footprint (14×14) with a large weight (9.4 MB), so both weight and input tiling are required but the small spatial size makes the tiling overhead negligible, ultimately pushing the kernel past the ridge point.

Source files: `conv.cu`, `conv_ref.h` | *Profiling script:* `run_ncu.sh` | *Roofline plot:* `plot_roofline.py`

CS259 Mini-Project 1 — Part 2: Single-Head Attention CUDA Kernels

1. Hardware and Theoretical Bounds

GPU: NVIDIA TITAN V (Volta GV100)

Parameter	Value
FP32 peak compute	14,900 GFLOPS (14.9 TFLOPS)
HBM2 peak bandwidth	652.8 GB/s
L2 cache	4.5 MB
Shared memory / SM	96 KB
SMs	80
Ridge point	22.82 FLOPs/byte

---

2. Parallelization Strategy (Q1)

Prefill kernels

Each query row i is independent, so the natural mapping is **one CUDA block per query row**: `Grid = (S,)`. Within a block, D=64 threads handle the D output dimensions in parallel — one thread owns one output element and accumulates its result across all attended keys.

For the flash kernels, the K/V tile (16 or 32 rows) is loaded cooperatively by all 64 threads into shared memory; each thread contributes `tile_rows` load operations strided by D. The cross-thread dot-product reduction uses `__shfl_down_sync` within each warp and a two-element `swarp[]` array for cross-warp communication.

Scalability:

- **Context length S**: Grid grows linearly with S, which is ideal. However, flash attention must re-read K and V for every query row; total DRAM traffic grows as $O(S^2)$, so the kernel transitions from compute-bound (S=4096) to memory-bound (S=65536) as S increases.
- **Batch size**: Not parallelized — the current implementation is single-batch. Adding a batch dimension would require an additional grid axis `Grid = (S, batch)` and proportionally more memory.

- **Head dimension D**: Hard-coded as 64 (one thread per dimension). Changing D would require restructuring the block and the warp-level reduction.

Decode kernels

Decode has only one query vector; the natural parallelism is across the C context positions. The flash decode kernels use **one block per slice of C**: `Grid = ([C/slice],)`, Block = D=64 threads. This distributes K/V reads across multiple SMs.

Scalability:

- **Context length C**: More blocks launch as C grows, improving SM utilization. For C=4096 (16 blocks), most SMs are idle; for C=65536 (256 blocks), all 80 SMs are engaged but at only ~10% occupancy — the problem is still too small to saturate the GPU.
- **Batch size**: Same limitation as prefill — not parallelized across batch.
- **Slice size**: Smaller slices launch more blocks (better SM coverage) but increase Phase 2 merge cost. Slice=256 was chosen to balance these effects.

---

3. Algorithmic FLOPs (Q2)

Prefill (causal, D=64)

Each output row i attends to keys j = 0, 1, …, i (causal masking). The computation for row i is:

1. **QK dot products**: i+1 keys × D FMAs = (i+1)·D FMAs
2. **Weighted V sum**: i+1 values × D FMAs = (i+1)·D FMAs

Total FMAs per row i = 2·(i+1)·D (the factor of 2 counts both QK and AV passes).
Summing over all rows and converting FMAs to FLOPs (1 FMA = 2 FLOPs):

```
FMAs = Σ_{i=0}^{S−1} 2·(i+1)·D
     = 2·D · S·(S+1)/2
     = D · S · (S+1)

FLOPs = FMAs × 2 = 2·D·S·(S+1)
```

The factor of 2 in the final answer comes from the FMA→FLOPs conversion (1 FMA = 1 multiply + 1 add = 2 FLOPs), *not* from double-counting QK and AV — those are already accounted for in the per-row FMA count.

Workload	S	FLOPs
Prefill S=4096	4,096	2 × 64 × 4096 × 4097 ≈ **2.15 GFLOP**
Prefill S=65536	65,536	2 × 64 × 65536 × 65537 ≈ **549.8 GFLOP**

Decode (D=64)

One query vector attends to all C context positions:

1. **QK dot products**: C keys × D multiply-adds = C·D FLOPs
2. **Weighted V sum**: C values × D multiply-adds = C·D FLOPs

Total = **2·C·D**. The softmax itself (exp, division) is not counted as FFMA but contributes to execution time.

Workload	C	FLOPs
Decode C=4096	4,096	2 × 4096 × 64 ≈ **0.52 MFLOP**
Decode C=65536	65,536	2 × 65536 × 64 ≈ **8.39 MFLOP**

Note: NCU counts 4·C·D FFMA instructions for decode (not 2·C·D) because the online softmax update — `s = s*corr + es` and `o = o*corr + es*sV` — adds roughly 2·C·D additional multiply-adds beyond the bare QK/AV math. The algorithmic FLOP count uses only the mathematically necessary operations.

---

4. Execution Time (Q3)

Kernel	Size	Time (ms)	GFLOPS	Speedup vs naive
prefill_naive	S=4096	7.356	292.0	1.00×
prefill_flash	S=4096	5.444	394.6	**1.35×**
prefill_flash_v2	S=4096	8.433	254.7	0.87×
prefill_naive	S=65536	—	—	N/A (OOM)
prefill_flash	S=65536	1406.6	390.8	—
prefill_flash_v2	S=65536	2128.0	258.4	0.66× vs flash
decode_naive	C=4096	0.417	2.52	1.00×
decode_flash	C=4096	0.109	9.66	**3.83×**
decode_flash_v2	C=4096	0.106	9.85	**3.91×**
decode_naive	C=65536	8.700	1.93	1.00×
decode_flash	C=65536	0.186	90.0	**46.8×**
decode_flash_v2	C=65536	0.183	91.5	**47.5×**

GFLOPS = algorithmic FLOPs / measured time.

---

5. Roofline Analysis (Q4)

Theoretical arithmetic intensity

Theoretical AI assumes each input matrix element is read exactly once and each output element is written once:

Minimum bytes (float32 = 4 bytes):

- Prefill: read Q (S·D·4) + read K (S·D·4) + read V (S·D·4) + write O (S·D·4) = 4·S·D·4 bytes
- Decode: read q (D·4) + read K (C·D·4) + read V (C·D·4) + write o (D·4) ≈ 2·C·D·4 bytes (q, o negligible)

Workload	Algorithmic FLOPs	Min bytes	Theoretical AI
Prefill S=4096	2.148×10^9	4 × 4096 × 64 × 4 = 4.194 MB	**512 FLOPs/byte**
Prefill S=65536	549.8×10^9	4 × 65536 × 64 × 4 = 67.1 MB	**8,192 FLOPs/byte**
Decode C=4096	0.524×10^6	2 × 4096 × 64 × 4 = 2.10 MB	**0.25 FLOPs/byte**
Decode C=65536	8.39×10^6	2 × 65536 × 64 × 4 = 33.6 MB	**0.25 FLOPs/byte**

Measured arithmetic intensity (from NCU)

Measured AI = (ffma_count × 2) / (dram_read + dram_write)

Kernel	Size	DRAM Read	DRAM Write	ffma count	Measured AI	Occupancy	DRAM BW%
prefill_naive	S=4096	33.88 MB	47.76 MB	1,149M	28.2 FLOPs/B	47.88%	1.66%
prefill_flash	S=4096	3.15 MB	1.19 MB	5,371M	2,475 FLOPs/B	29.71%	0.12%
prefill_flash_v2	S=4096	3.15 MB	1.20 MB	5,371M	2,469 FLOPs/B	14.57%	0.08%
prefill_flash	S=65536	742.72 GB	18.21 MB	1,374B	3.70 FLOPs/B	34.08%	80.90%
prefill_flash_v2	S=65536	11.56 GB	18.21 MB	1,374B	237 FLOPs/B	15.57%	0.83%
decode_naive	C=4096	2.10 MB	0.51 KB	561K	0.53 FLOPs/B	7.30%	0.60%
decode_flash	C=4096	2.10 MB	1.28 KB	2,621K	2.49 FLOPs/B	3.12%	2.64%

Kernel	Size	DRAM Read	DRAM Write	ffma count	Measured AI	Occupancy	DRAM BW%
decode_flash_v2	C=4096	2.11 MB	1.02 KB	2,621K	2.48 FLOPs/B	3.12%	2.93%
decode_naive	C=65536	33.96 MB	1.90 MB	8,978K	0.50 FLOPs/B	7.30%	0.63%
decode_flash	C=65536	33.56 MB	1.44 MB	41,943K	2.40 FLOPs/B	9.89%	38.29%
decode_flash_v2	C=65536	33.56 MB	1.44 MB	41,943K	2.40 FLOPs/B	9.96%	40.07%

Theoretical vs measured AI — what the difference reveals

Kernel	Theoretical AI	Measured AI	Ratio	Explanation
prefill_naive S=4096	512 FLOPs/B	28.2 FLOPs/B	18× lower	Writes S×S score buffer (81 MB) not in theoretical min-bytes
prefill_flash S=4096	512 FLOPs/B	2,475 FLOPs/B	**4.8× higher**	Online softmax adds ~4.7× extra FFMA vs bare QK+AV; data cached in L2
prefill_flash S=65536	8,190 FLOPs/B	3.70 FLOPs/B	2,200× lower	K/V re-read S/2 times each on average; total DRAM = 742 GB not 67 MB
decode_naive C=65536	0.25 FLOPs/B	0.50 FLOPs/B	2× higher	Writes score buffer (C×4 bytes) beyond minimum; slightly more FFMA from softmax
decode_flash C=65536	0.25 FLOPs/B	2.40 FLOPs/B	**9.6× higher**	Online softmax FFMA inflates numerator; DRAM close to minimum

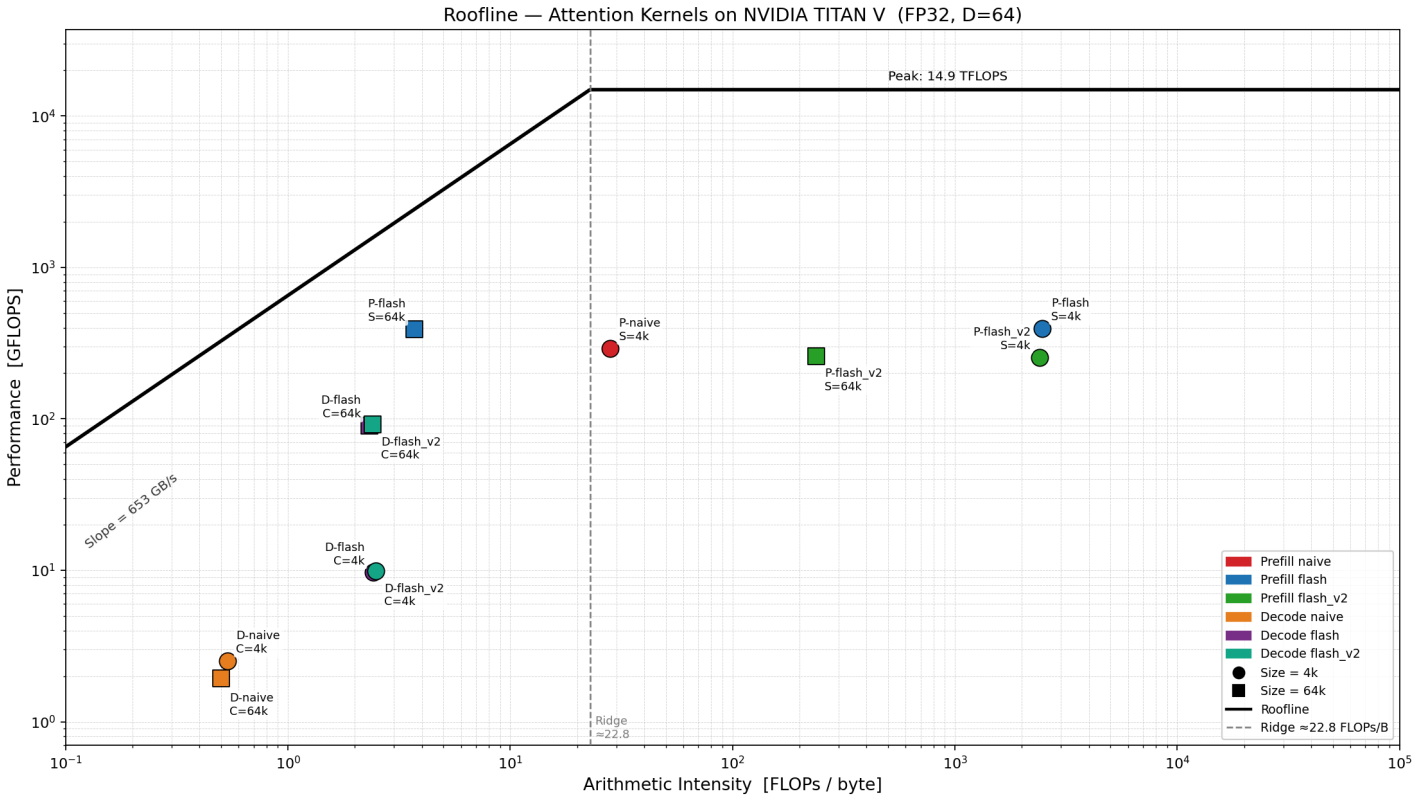
Key observations:

- **prefill_naive**: measured AI far below theoretical because the score buffer write (47.76 MB) is not in the theoretical minimum — theory assumes one read/write per element, but naive writes S^2 scores to DRAM.
- **prefill_flash S=4096**: measured AI far *above* theoretical because (a) online softmax multiplies instruction count by ~4.7× and (b) all data fits in L2, so DRAM denominator is near minimum while FFMA numerator is large.
- **prefill_flash S=65536**: the single largest discrepancy (2,200×). Theoretical AI assumes K/V are each read once (67 MB), but the causal pattern forces re-reading every key for every later query, accumulating

742 GB of DRAM traffic. This is the signature of working-set overflow from HBM.

- **decode**: measured AI is higher than theoretical mainly due to the online softmax FFMA overhead. Both naive and flash read approximately the theoretical minimum DRAM bytes.

Roofline placement and bound identification



Kernel	Measured AI	Ridge = 22.82	Bound	% of roofline ceiling
prefill_naive S=4096	28.2 FLOPs/B	> ridge	compute	292 / 14900 = **2.0%**
prefill_flash S=4096	2,475 FLOPs/B	>> ridge	compute	395 / 14900 = **2.7%**
prefill_flash_v2 S=4096	2,469 FLOPs/B	>> ridge	compute	255 / 14900 = **1.7%**
prefill_flash S=65536	3.70 FLOPs/B	< ridge	**memory**	391 / (652.8×3.70) = **16.2%**
prefill_flash_v2 S=65536	237 FLOPs/B	>> ridge	compute	258 / 14900 = **1.7%**
decode_naive C=4096	0.53 FLOPs/B	<< ridge	**memory**	2.52 / (652.8×0.53) = **0.73%**
decode_flash C=4096	2.49 FLOPs/B	<< ridge	**memory**	9.66 / (652.8×2.49) = **0.59%**

Kernel	Measured AI	Ridge = 22.82	Bound	% of roofline ceiling
decode_flash_v2 C=4096	2.48 FLOPs/B	<< ridge	**memory**	9.85 / (652.8×2.48) = **0.61%**
decode_naive C=65536	0.50 FLOPs/B	<< ridge	**memory**	1.93 / (652.8×0.50) = **0.59%**
decode_flash C=65536	2.40 FLOPs/B	< ridge	**memory**	90 / (652.8×2.40) = **5.7%**
decode_flash_v2 C=65536	2.40 FLOPs/B	< ridge	**memory**	91.5 / (652.8×2.40) = **5.8%**

All kernels are far from their respective ceilings. The potential for further optimization depends on the bound:

- **Compute-bound kernels** (prefill S=4096): the bottleneck is the online softmax serial dependency chain (m → corr → es → s → o), which prevents instruction-level pipelining. Occupancy is also limited by shared memory. Further gains would require software pipelining or FP16/tensor-core math to increase throughput.
- **Memory-bound kernels** (prefill S=65536, all decode): the bottleneck is HBM bandwidth. For prefill at large S, the only path forward is reducing total DRAM traffic — e.g., by processing multiple query rows per block to increase K/V reuse. For decode, the problem is too small to saturate the GPU even at 256 blocks; larger batch sizes or longer context lengths would help.

---

6. Implementation Overview (Q5 context)

6.1 Prefill Kernels

`prefill_naive` (baseline, S ≤ 4096 only)

- Grid: (S,) — one block per query row. Block: 256 threads.
- Allocates an S×S global score buffer. For S=4096 this is 67 MB; for S=65536 it would require 17.2 GB, exceeding TITAN V's 12 GB HBM — this kernel cannot be launched for large S.
- Three passes: (1) dot-products → score buffer, (2) global max + exp/sum for softmax, (3) weighted V accumulation.

`prefill_flash` (tile = 16)

- Implements Flash Attention with online softmax, eliminating the score buffer entirely.
- Grid: (S,), Block: D=64 threads — one thread per output dimension.
- Loads K/V in tiles of 16 rows into shared memory. Score reduction uses `__shfl_down_sync` (intra-warp) and `swarp[2]` (cross-warp). Running state (m, sum, out) updated per key using: m_new = max(m, score); corr = exp(m − m_new); s = s·corr + exp(score − m_new).

`prefill_flash_v2` (tile = 32)

- Identical to `prefill_flash` but doubles tile size to 32 rows, halving outer-loop iterations and `__syncthreads()` barriers.
- Shared memory doubles from ~8 KB to ~16 KB per block (occupancy penalty, see Section 7).

6.2 Decode Kernels

`decode_naive` (baseline)

- Grid: (1,), Block: 256 threads. Global score buffer of size C.
- Three-pass softmax. Single block → single SM → 79 SMs idle.

`decode_flash` (2-phase Flash Decoding, slice = 256)

- Phase 1: ⌈C/256⌉ blocks, each processing 256 context positions with online softmax, writing (m, d, O_partial) to global.
- Phase 2: single block merges all partials using online softmax merge: m_new = max(m, m_b); rescale and accumulate.

`decode_flash_v2` (slice = 256, float4 loads)

- Identical to `decode_flash` but replaces scalar loads with 128-bit `float4` loads, reducing load instruction count.

---

7. Analysis

7.1 Prefill — necessity of Flash Attention

Standard attention for S=65536 requires $65536^2 \times 4$ bytes = **17.2 GB** for the score matrix, exceeding TITAN V's 12 GB HBM. Flash Attention eliminates this by maintaining only O(tile·D) shared memory state regardless of S.

7.2 Prefill — occupancy from first principles

Occupancy is constrained by shared memory per SM (96 KB):

Kernel	smem / block	Max blocks / SM	Threads / SM	Theoretical occupancy
prefill_flash	8,208 B	⌊96 KB / 8208 B⌋ = 11	11 × 64 = 704	704 / 2048 = **34.4%**
prefill_flash_v2	16,400 B	⌊96 KB / 16400 B⌋ = 5	5 × 64 = 320	320 / 2048 = **15.6%**

NCU confirms: flash = 29.7%, flash_v2 = 14.6% — matching the smem-limited prediction. Doubling the tile halves occupancy.

7.3 Prefill — compute-bound vs memory-bound

S=4096 — compute-bound

Both flash kernels: measured AI ≈ 2,470 FLOPs/byte >> ridge (22.82). DRAM throughput < 0.12%.

- `prefill_flash`: sm__throughput = 57.5%, occupancy = 29.7% → 395 GFLOPS. Bottleneck: online softmax serial dependency (m → corr → es → s → o) prevents ILP.
- `prefill_flash_v2`: occupancy = 14.6%, sm__throughput = 36.3% → 255 GFLOPS. The fewer syncthreads barriers cannot compensate for halved warp count.

S=65536 — memory-bound despite identical algorithm

`prefill_flash`: DRAM throughput = 80.9% (≈528 GB/s), DRAM read = 742.72 GB. Measured AI = 3.70 FLOPs/byte (below ridge). Key j is read by queries j through S−1, averaging S/2 ≈ 32,768 re-reads; total traffic ≈ 1.1 TB >> 4.5 MB L2.

Why DRAM BW = 80.9% but roofline efficiency = only 16.2%: Causal load imbalance — block i processes i+1 keys. Early blocks (small i) finish almost instantly and their SMs go idle; late blocks dominate runtime. The 80.9% DRAM figure reflects only the final waves of large blocks, not the average across the full kernel. The roofline model assumes uniform load distribution, so the gap is a direct measure of the imbalance penalty.

S=65536, flash_v2 — tradeoff

DRAM read drops 64× (11.56 GB vs 742 GB) due to better L2 reuse from the larger tile. Yet performance is worse (258 vs 391 GFLOPS): occupancy drops from 34% to 16%, sm__throughput falls from 56.8% to 37.0%. Cache efficiency gains are outweighed by occupancy loss.

7.4 Decode — multi-block parallelism

All decode kernels: AI ≈ 0.5–2.5 FLOPs/byte << ridge. Memory-bound.

decode_naive: DRAM throughput = 0.63% with single block — only one SM issues memory requests, leaving 99.4% of HBM bandwidth idle.

decode_flash: 256 blocks → DRAM throughput rises to 38.3% → **46.8× speedup**. Hundreds of concurrent memory requests better utilize HBM.

decode_flash_v2 (float4): Same 256 blocks. L1-texture traffic increases from 33.70 MB to 98.71 MB (wider vector fetches), DRAM throughput rises to 40.1% → 91.5 vs 90.0 GFLOPS.

Root cause of 40% ceiling: 256 blocks × 64 threads = 16,384 threads vs GPU capacity of 163,840. Only ~10% occupancy — the problem is simply too small to saturate the hardware.

8. Optimizations (Q5)

Optimization	Workload	Expected benefit	Actual outcome	Root cause
Flash Attention (no score buffer)	Prefill S=65536	Enables large-S	**Essential** (naive OOM)	Eliminates $O(S^2)$ buffer
Flash Attention	Prefill S=4096	Reduce DRAM traffic	**+35% speedup** (1.35×)	DRAM traffic 81 MB → 4.3 MB
Larger tile 16→32 (flash_v2)	Prefill S=4096	Fewer syncthreads	**Slower** (0.87×)	smem 8→16 KB; occupancy 34%→15%; sm__throughput 57%→36%
Larger tile 16→32 (flash_v2)	Prefill S=65536	Better L2 cache reuse	**Slower** (0.66×)	64× less DRAM but occupancy penalty dominates
Multi-block flash decode	Decode C=65536	More SM utilization	**+47× speedup**	DRAM BW% 0.63%→38%; all 80 SMs engaged
float4 loads (flash_v2)	Decode C=65536	Wider memory transactions	**+1.7%** (marginal)	Both kernels read identical DRAM bytes; float4 marginally reduces L1 pressure
float4 loads (flash_v2)	Decode C=4096	Same as above	**Negligible**	Problem too small; GPU idle time dominates

Most impactful: Flash Attention for prefill (eliminates OOM for large S; 1.35× for small S) and multi-block Flash Decoding (47× speedup by utilizing all SMs).

Least impactful / negative: Larger tile (flash_v2) — consistently slower because the shared memory occupancy penalty outweighs both the barrier reduction and cache reuse benefits. This reveals that for kernels with serial dependency chains, **occupancy is the primary throughput lever**, not cache efficiency.

---

9. Conclusion

Flash Attention is indispensable for large prefill sequences: standard attention requires 17.2 GB for S=65536, exceeding hardware limits, while flash attention operates with O(tile·D) shared memory regardless of S. At S=4096, flash attention achieves a **1.35× speedup** (DRAM traffic 81.6 MB → 4.3 MB; compute-bound at 2.7% of FP32 peak). At S=65536, the bottleneck shifts to HBM bandwidth (80.9% utilization) because K/V must be re-read for each query row, accumulating 742 GB of DRAM traffic despite each matrix being only 16 MB; causal load imbalance further limits roofline efficiency to 16.2%.

Attempts to improve prefill via a larger tile (flash_v2) revealed a fundamental tradeoff: 2× tile → 64× less DRAM traffic but half the occupancy due to doubled shared memory, resulting in net slowdown. This shows that for kernels with serial dependency chains, occupancy — not cache efficiency — is the primary throughput determinant.

For decode, multi-block Flash Decoding provides a **47× speedup** by distributing the KV cache across 256 blocks and raising DRAM throughput from 0.6% to 40%. Both tasks remain far from theoretical limits; further improvements would require breaking the online softmax dependency chain (warp-level software pipelining) or tensor-core FP16/BF16 computation.